Mini Search Engine Using Data Structures and Information Retrieval



B.Siddhartha-2500090225 C.V.JaswanthReddy-2520080025

Abstract

The Mini Search Engine Using Data Structures and Information Retrieval is a software application designed to search and retrieve relevant information from a collection of text documents. The system reads multiple text files, processes their contents, and creates an index of the words found in each document. When a user enters a keyword, the system searches this index and quickly identifies the documents containing the requested word. The project uses important Data Structures and Algorithms, particularly HashMap for efficient indexing and searching, ArrayList for storing documents and search results, and HashSet for avoiding duplicate entries. The system can also count the occurrence of searched words and rank the matching documents based on their relevance.

Introduction

The Mini Search Engine Using Data Structures and Information Retrieval is a simple search system designed to find relevant information from a collection of text documents. As the number of digital documents increases, manually searching for required information becomes time-consuming. This project provides a faster and more organized way to search for keywords within multiple documents.

The system reads the available text files, processes their contents, and creates an index that connects each word with the documents in which it appears. When a user enters a keyword, the system uses this index to quickly find and display the matching documents instead of checking every document individually.

Software&Hardware Requirements

Software Requirements:
Operating System: Windows 10/11
Programming Language: Java
JDK: JDK 17 or above
IDE: VS Code / IntelliJ IDEA / Eclipse
Data Structures: HashMap, ArrayList, HashSet
File Handling: Java File I/O
Database: Not Required

Hardware Requirements
Processor: Intel Core i3 or equivalent
RAM: 4 GB or above
Storage: 500 MB free space
Display: 1366 × 768 resolution or higher
Keyboard: Standard keyboard
Mouse: Standard mouse
Computer/Laptop: Any modern computer or laptop

MINI SEARCH ENGINE

Using Data Structures and Information Retrieval

DOCUMENTS

Java.txt

Python.txt

DSA.txt

...

PREPROCESSING

- Read Documents
- Tokenization & Normalization
- Remove Stop Words
- Stemming / Lemmatization

INVERTED INDEX (HashMap)

WORD	DOCUMENTS
java	Java.txt, DSA.txt
python	Python.txt
data	DSA.txt
structure	DSA.txt
algorithm	DSA.txt
easy	Java.txt, Python.txt
object	Java.txt
...	...

USER SEARCH

java

SEARCH RESULTS

Keyword: java

Found in 2 document(s)

1. Java.txt
2. DSA.txt



HOW IT WORKS

Collection of Documents → Process and Extract Words → Build Index using **HashMap** → User Searches Keyword → Retrieve Matching Documents

Literature Survey

- Information Retrieval Systems: Previous studies have explored methods for efficiently storing, indexing, and retrieving information from large collections of documents.
- Inverted Indexing: Research shows that inverted indexes are widely used in search systems to map keywords to the documents in which they occur, allowing faster keyword-based retrieval.
- Hashing Techniques: Hash-based data structures such as HashMap provide efficient access to stored information and are useful for creating keyword indexes.
- Text Processing: Various information retrieval systems use text preprocessing techniques such as tokenization, case normalization, and removal of unnecessary characters to improve search accuracy.
- Search and Ranking: Existing systems commonly rank retrieved documents according to factors such as keyword frequency and relevance so that the most useful results appear first.
- Proposed Approach: Based on these concepts, the proposed Mini Search Engine combines text processing, inverted indexing, hashing, searching, and ranking to create a simple and efficient document retrieval system suitable for small collections of text files.

Methodology

1. Document Collection - Collect multiple text files as input documents.
2. Document Reading - Read the contents of each text file using Java file handling.
3. Text Preprocessing - Convert the text to lowercase and remove unnecessary punctuation and special characters.
4. Word Extraction - Split the processed text into individual words.
5. Index Creation - Create an inverted index using HashMap to map each unique word to the documents containing it.
6. User Search - Accept a keyword as input from the user.
7. Search and Retrieval - Search the index and retrieve the documents containing the entered keyword.
8. Result Ranking - Rank the retrieved documents based on the frequency of the searched keyword.
9. Result Display - Display the matching documents and their relevance to the user.